



Relaciones Unidireccionales en JPA

Cómo conectar entidades en una
sola dirección

¿Qué es una relación unidireccional?



Una sola dirección

Solo una entidad "conoce" a la otra.



Atributo de referencia

Solo una clase contiene el atributo que referencia a la otra.



Simplicidad en el diseño

Ayuda a reducir la complejidad y el acoplamiento.

Imagina una flecha de A hacia B ($A \rightarrow B$); solo A sabe de B.

Tipos de relaciones unidireccionales comunes



@OneToOne

Una entidad A se relaciona con una entidad B.



@ManyToOne

Varias entidades A se relacionan con una entidad B.



@OneToMany

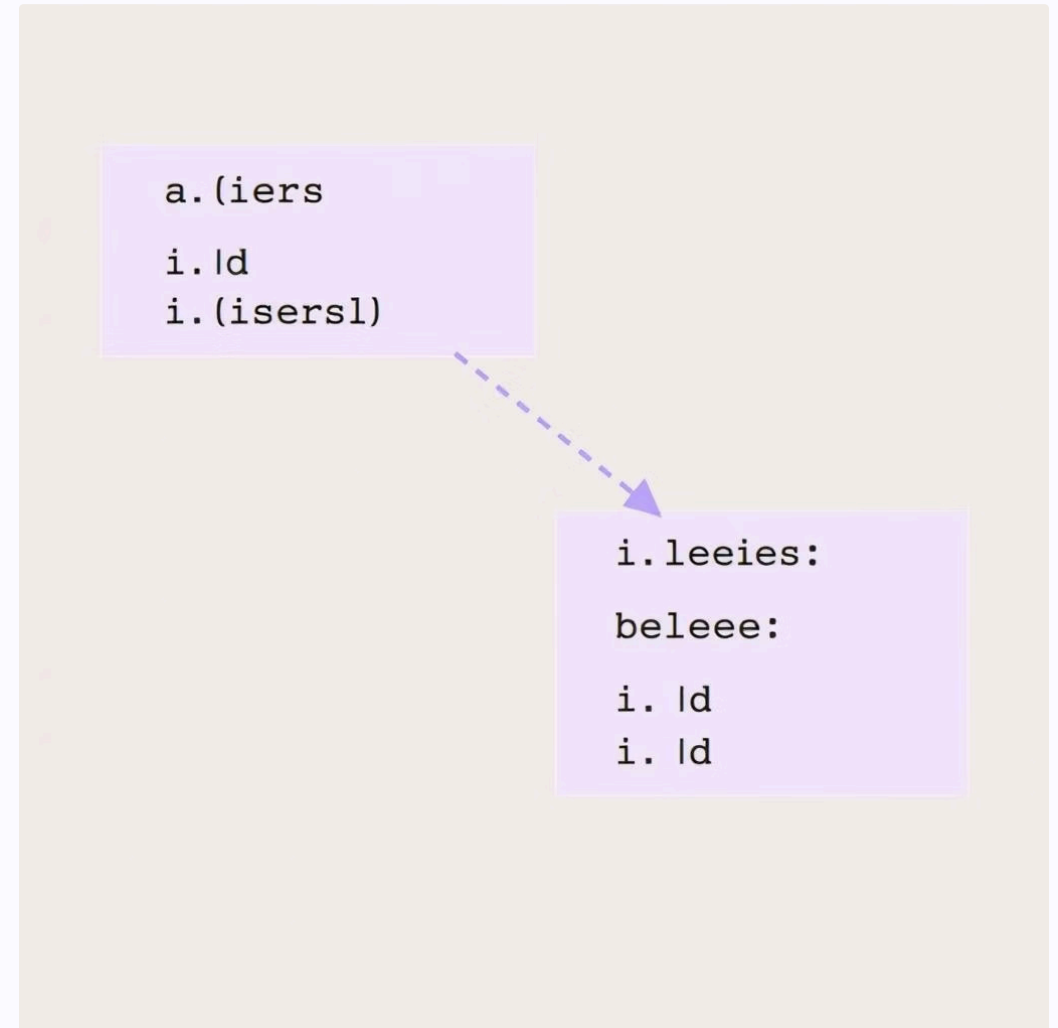
Una entidad A se relaciona con varias entidades B.

Nota: La relación @ManyToMany suele implementarse de forma bidireccional, lo veremos más adelante.

Ejemplo Visual: @OneToOne

Usuario y DetallesUsuario

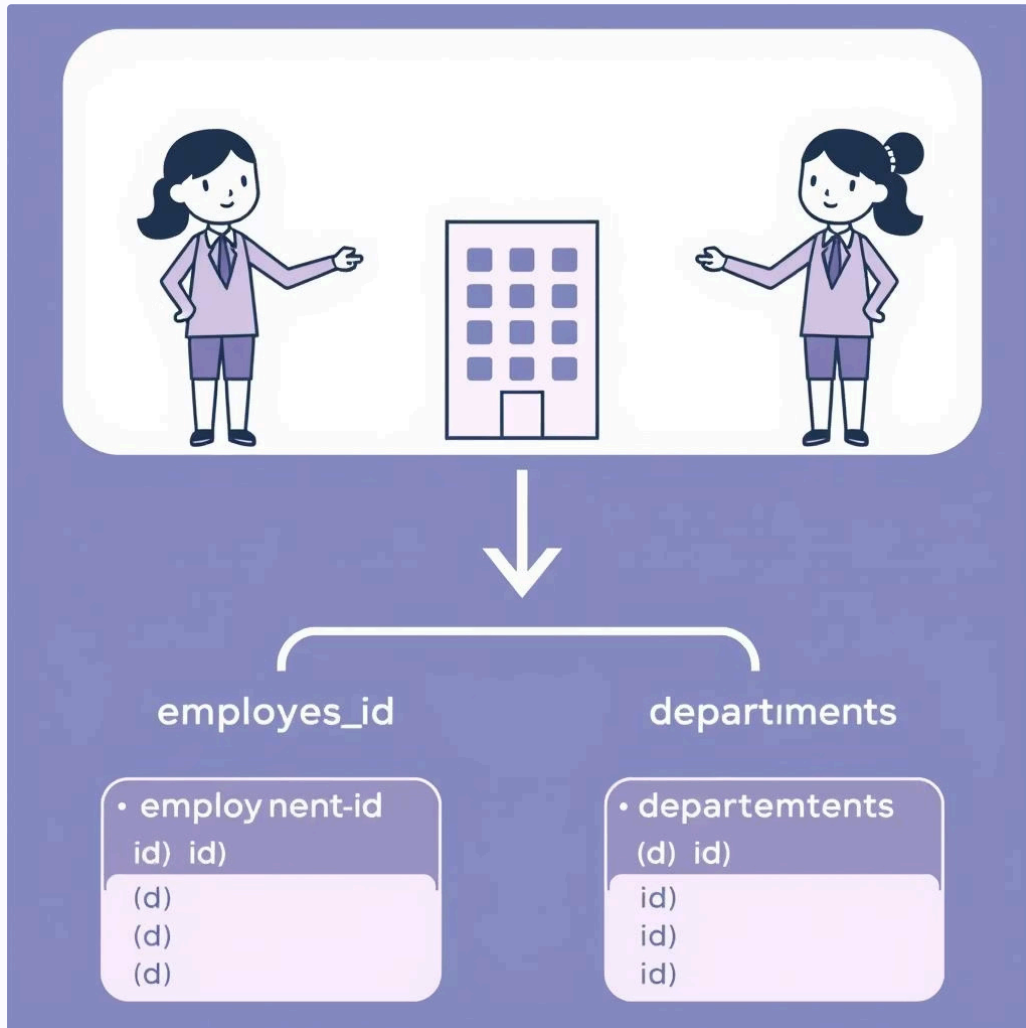
- **Usuario** contiene el atributo de DetallesUsuario y la clave foránea (detalles_id).
- **DetallesUsuario** no tiene referencia a Usuario.
- La clave foránea en la tabla usuarios es **única**.



Ejemplo Visual: @ManyToOne

Empleados y Departamento

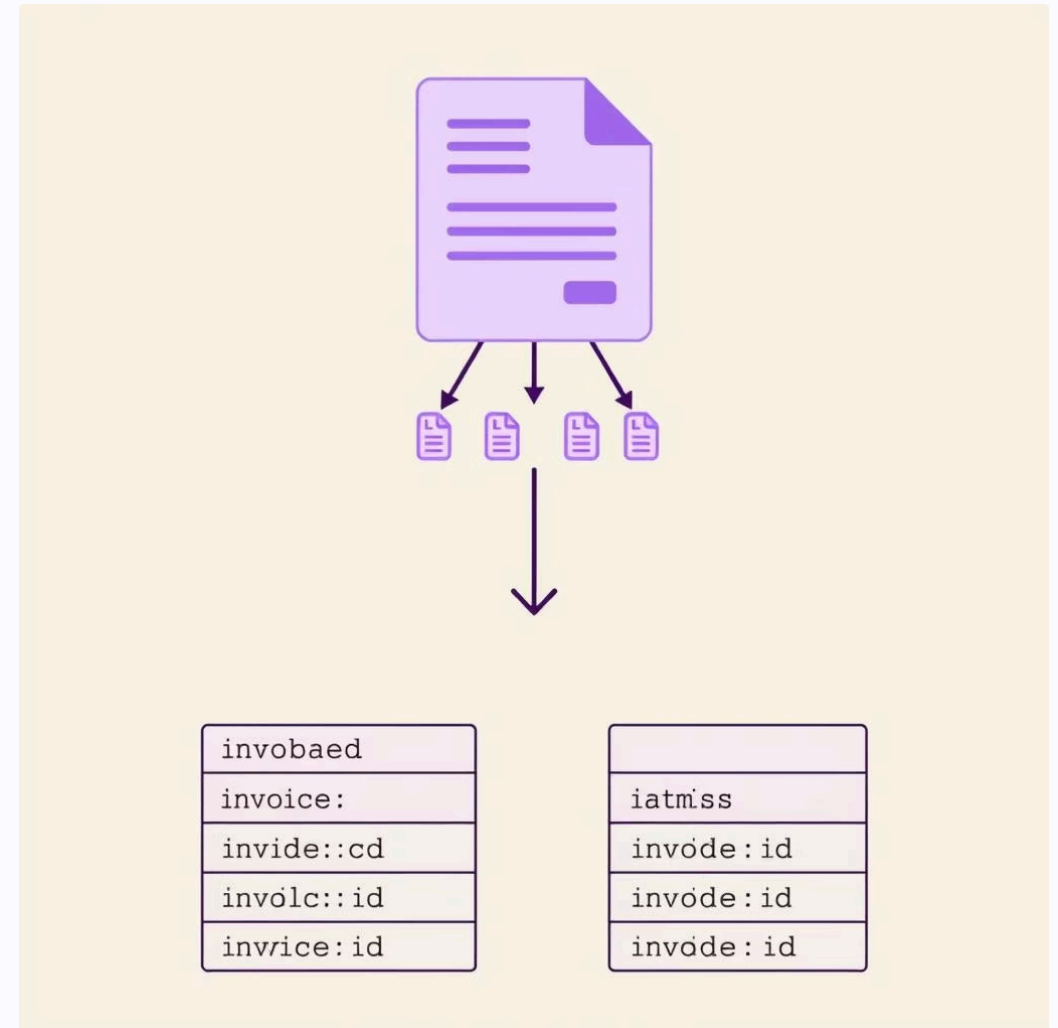
- Múltiples empleados se asocian a un **único departamento**.
- La clave foránea (`departamento_id`) se encuentra en la tabla `empleados`.



Ejemplo Visual: @OneToMany

Factura y Ítems de Factura

- Una **Factura** contiene una lista de **ItemFactura**.
- Los **Ítems de Factura** no necesitan conocer a la **Factura** padre.
- La clave foránea (factura_id) se define usando `@JoinColumn` en la tabla de ítems.



Código Simplificado: @OneToOne, @ManyToOne, @OneToMany

@OneToOne

```
@OneToOne
@JoinColumn(name =
"detalles_id", unique = true)
private DetallesUsuario detalles;
```

La FK se crea en la tabla de la entidad propietaria (Usuario).

@ManyToOne

```
@ManyToOne
@JoinColumn(name =
"departamento_id")
private Departamento
departamento;
```

La FK (departamento_id) está en la tabla Empleado.

@OneToMany

```
@OneToMany
@JoinColumn(name =
"factura_id")
private List<ItemFactura> items;
```

Sin mappedBy, es unidireccional.
@JoinColumn previene tabla intermedia.

Buenas Prácticas en Unidireccionales

FetchType.LAZY

Siempre que sea posible, usar para evitar cargas innecesarias.

mappedBy

No usar `mappedBy` en relaciones unidireccionales.

@JoinColumn

Fundamental para controlar la columna de la clave foránea.

Cascade y orphanRemoval

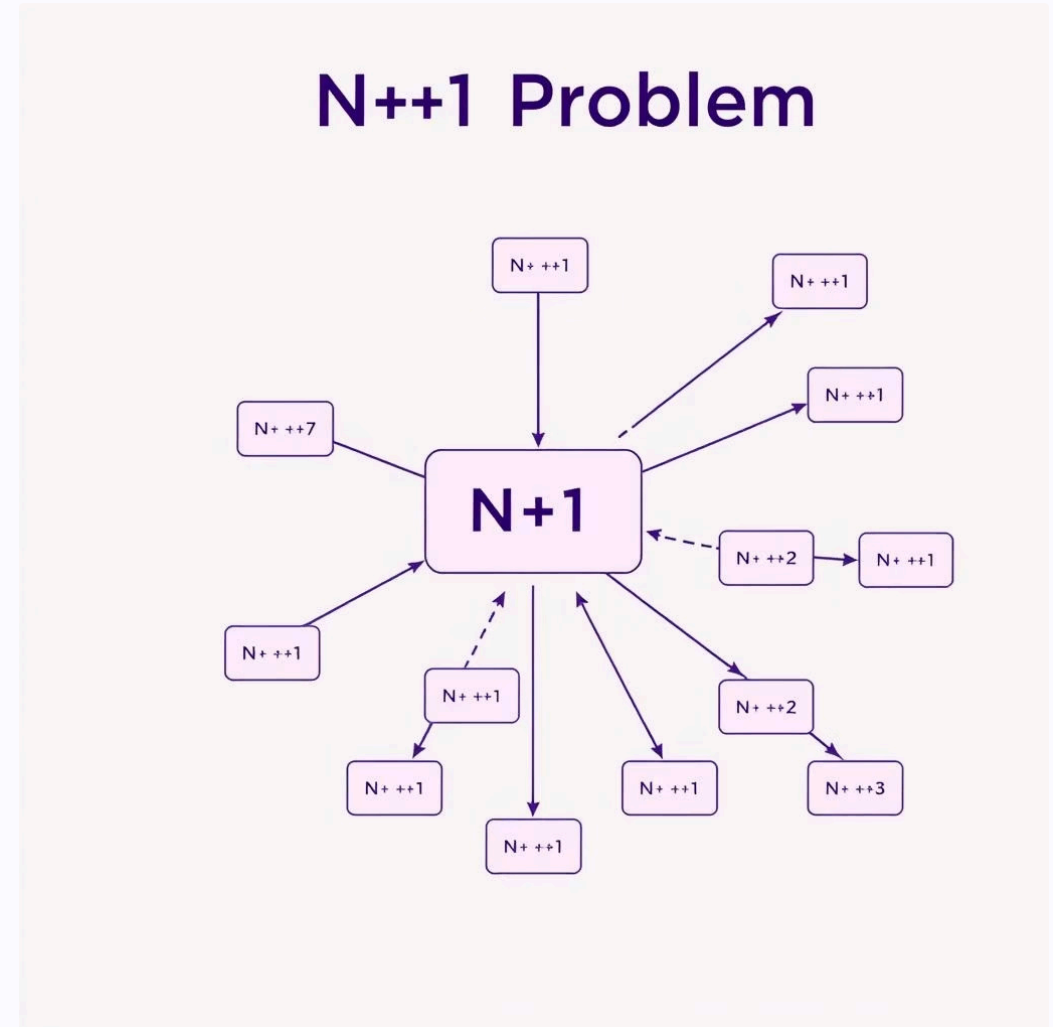
Aplicar con precaución, solo cuando sea estrictamente necesario.

Problema Común: N+1

¿Qué es?

Una consulta inicial puede generar **N consultas adicionales** para cargar las relaciones asociadas.

- Típico con `FetchType.EAGER`.
- Impacta gravemente el rendimiento.



Solución:

- `JOIN FETCH` para cargar las relaciones en una sola consulta.
- `EntityGraph` para definir gráficos de carga dinámicos.

Resumen Final

Un Solo Lado

Solo una entidad conoce la relación.

Tipos Clave

@ManyToOne, @OneToOne, @OneToMany.

FK Controlada

Siempre con @JoinColumn.

Carga Perezosa

Preferir LAZY para listas y relaciones grandes.

Próximo Video: Relaciones Bidireccionales en JPA

Cómo lograr sincronización en ambos sentidos sin errores.